



AULA 3

Boas práticas de orientação a objetos e de codificação em Python



Objetivos de aprendizagem da aula

Ao final desta aula, você irá:

- Compreender os conceitos dos princípios SOLID e saber como utilizá-los nos seus códigos.
- Reconhecer e saber como aplicar o guia de estilos Python.
- Entender o conceito de *clean code*.
- Entender como utilizar boas práticas de codificação em Python.



Nesta terceira aula, vamos estudar assuntos que irão “profissionalizar” os nossos códigos.

Primeiro, falaremos dos princípios SOLID, cinco princípios aplicados a orientação a objetos (OO) que visam a facilitar a manutenção do código. Em seguida, conheceremos o guia de estilos Python e o conceito de *clean code*. Além disso, falaremos de boas práticas de codificação em Python.

Que história é essa?

Princípios SOLID

Os princípios SOLID têm como objetivo ajudar a realizar um bom projeto de OO para um *software* de código flexível, escalável, sustentável e reutilizável. SOLID é um acrônimo dos cinco princípios:

- *Single responsibility principle* (SPR) – Princípio de responsabilidade única
- *Open/closed principle* (OCP) – Princípio do aberto/fechado
- *Liskov substitution principle* (LSP) – Princípio da substituição de Liskov
- *Interface segregation principle* (ISP) – Princípio de segregação de interface
- *Dependency inversion principle* (DIP) – Princípio da inversão de dependência

Vamos conhecer melhor cada um deles?



Clique em cada *box* para conhecer

Interativo

Descrição do interativo

Guia de Estilos

Com a finalidade de manter seu código organizado e fácil para que outros programadores (ou até você mesmo) possam compreender e dar manutenção, recomenda-se que um bom programador siga boas práticas para escrever seus códigos. Com o objetivo de concentrar práticas recomendadas para escrever e manter códigos de forma organizada, o criador do Python - Guido Von Rossum - é o principal autor do guia de estilos da linguagem Python.



Mas como utilizar o guia de estilos corretamente?

Embora a existência de um guia de estilos para uma linguagem de programação seja uma excelente prática, pode ser difícil para os programadores recordarem e aplicarem corretamente todas essas regras.

Por isso, para ajudar nessa tarefa repetitiva e passível de erros humanos, a **comunidade Python** criou o **gui de estilos**.

Interativo

Descrição do interativo

Conforme descrito na PEP-1, as PEPs podem ser:

- **de padrões** – descrevem novos recursos ou implementações de Python;
- **informacionais** – problemas de *design* ou orientações gerais para a comunidade;
- **de processos** – processos do Python.

Aprenda mais

É importante destacar que todas as PEPs são indexadas pela PEP-0. Para saber mais sobre ela, [clique aqui](#).

Além da PEP-0, algumas outras PEPs são interessantes. Acompanhe.

PEP-8

PEP-20



Trat
a fo
um

Interativo

Descrição do interativo

Layout de códigos

O primeiro pensamento do Zen do Python é *Beautiful is better than ugly*, ou seja, bonito é melhor que feio. Isso significa que a formatação de um código está diretamente relacionada ao seu entendimento.



Clique para saber quais são as orientações sobre *layout* de código da PEP-8:

Indentação

Devem-se usar quatro espaços (ou um tab) por nível de indentação, sendo os espaços a forma

Interativo

Descrição do interativo

Nomenclatura

O segundo pensamento do Zen do Python é *Explicit is better than implicit*, em português explícito é melhor que implícito. Isso significa que a clareza quanto ao que queremos desenvolver em um bloco de código é muito importante, o que inclui nomear as variáveis, as funções, as classes e os pacotes de forma clara e consistente.



As orientações sobre nomenclatura na PEP-8 são resumidas na tabela a seguir:

Tipo	Convenção de nomenclatura	Exemplos
Função	Use uma palavra ou palavras minúsculas separadas por sublinhados (<code>_</code>)	<code>function</code> , <code>my_function</code>
Variável	Use uma letra, palavra ou palavras minúsculas separadas por sublinhados (<code>_</code>)	<code>x</code> , <code>var</code> , <code>my_variable</code>
Classe	Comece cada palavra com uma letra maiúscula (<i>CamelCase</i>)	<code>Model</code> , <code>MyClass</code>
Método	Use uma palavra ou palavras minúsculas separadas por sublinhados (<code>_</code>)	<code>class_method</code> , <code>method</code>
Constante	Use uma letra, palavra ou palavras maiúsculas separadas por sublinhados (<code>_</code>)	<code>CONSTANT</code> , <code>MY_CONSTANT</code> , <code>MY_LONG_CONSTANT</code>
Módulo	Use uma palavra ou palavras curtas e minúsculas separadas por sublinhados (<code>_</code>)	<code>module.py</code> , <code>my_module.py</code>
Pacote	Use uma palavra ou palavras curtas com todos os caracteres em minúsculas, sem separá-las por sublinhados	<code>package</code> , <code>mypackage</code>



Observação: evite utilizar letras únicas como `l` ou `O` para definir uma variável, pois podem ser facilmente confundidas com `1` ou `0`, por exemplo:

```
1 0 = 7 # isto pode parecer que estamos tentando atribuir 7 a zero
```

Comentários

Outro pensamento do Zen do Python relacionado ao guia de estilos é *If the implementation is hard to explain, it's a bad idea*, ou seja, se a implementação é difícil de explicar, é uma má ideia.

Isso reforça a importância de o programador sempre documentar de forma objetiva e precisa o seu código, o que facilita a sua manutenção e interoperabilidade.

Algumas recomendações sobre comentários de código listadas na PEP-8 são:

As linhas de comentários e *docstrings* devem ter no máximo 72 caracteres. Além disso, é importante **utilizar frases completas para comentários**, sempre começando com uma letra maiúscula.

!

Lembre-se de **atualizar os comentários sempre que alterar o seu código**.

Use blocos de comentário para documentar as seções do seu código e, dessa forma, ajudar outros programadores a compreenderem seu propósito e funcionalidade.

Mas lembre-se de observar as seguintes orientações:

- Utilize nos comentários a mesma indentação do código que eles descrevem.
- Inicie cada linha com uma tralha (#) seguida por um único espaço.
- Separe os parágrafos por uma linha contendo uma única #.

Observe o exemplo:

cod.py

```
1 def calcular_equacao_quadratica(a, b, c, x):
2     # Calcular a solução para uma equação quadrática
3     #
4     # Há sempre duas soluções para uma equação quadrática, x_1 e x_2
5     x_1 = (-b+(b**2-4*a*c)**(1/2))/(2*a)
6     x_2 = (-b-(b**2-4*a*c)**(1/2))/(2*a)
7     return x_1, x_2
```

#

Comentários *inline* são os escritos na mesma linha do seu respectivo código e são úteis para explicar o que faz determinada linha de código, porém devem ser usados com moderação e separados do código por pelo menos dois espaços. **Eles devem ser iniciados com uma # seguida por um único espaço**, assim como os comentários de blocos.

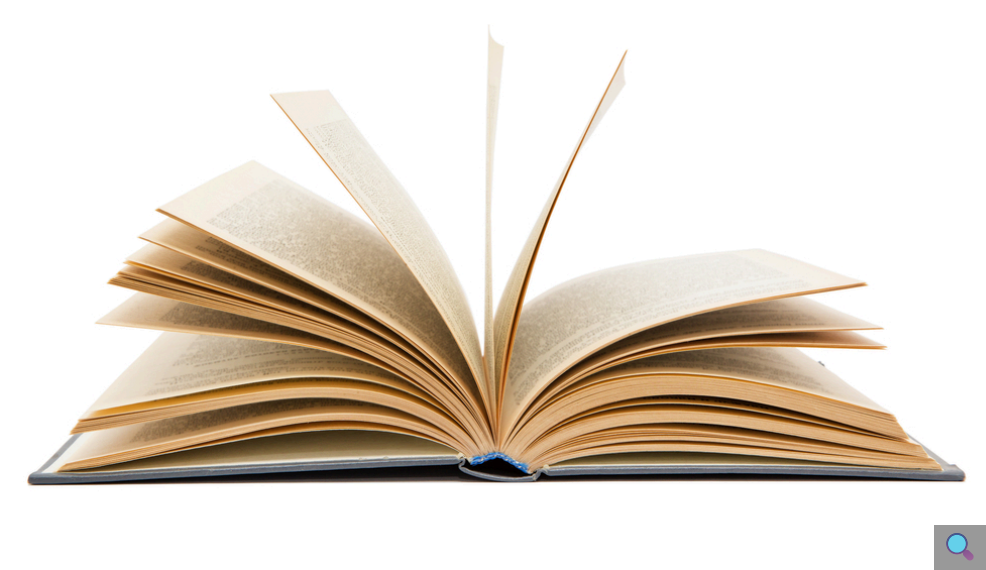
Não use comentários para explicar o óbvio, como no exemplo a seguir:

cod.py

```
1 idade = 7 # este é um comentário inline
2 nome = 'Tatiana Escovedo' # nome do estudante
```

Clean code

A expressão *clean code*, em português “código limpo” se popularizou com o livro de Robert C. Martin *Clean Code: A Handbook of Agile Software Craftsmanship*, publicado em 2008.



Este livro reforça que mesmo um código ruim pode funcionar, mas ele terá, provavelmente, legibilidade ruim, aumentando sua complexidade e custo de manutenção. Assim, **as boas práticas de um código limpo consistem em instruções direcionadas ao modo como escrevemos nosso código**, incluindo as técnicas identificadas como as melhores para realizar determinada tarefa.

Que tal conhecer essas boas práticas direcionadas para códigos em Python, que complementam ou reforçam as orientações de guia de estilos apresentadas anteriormente?

Veja na imagem a seguir clicando na seta para explorar o assunto:

Nomes significativos

Algumas regras simples para a criação de bons nomes e funções, parâmetros, classes, pacotes e arquivos-fonte

Usar nomes que revelem seu propósito: se o nome não revela seu propósito.

Interativo

Descrição do interativo

Além dessas boas práticas, há outras muito importantes para você implementar no seu dia a dia.

Sempre pergunte o que você quer saber de forma clara.

Utilize booleanos de forma implícita.

Tenha cuidado

Não é uma boa prática perguntas invertidas. Veja exemplo:

```
1 # Ruim
2 def lista_usuarios():
3     if request not in ('POST', 'PUT'):
4         return get_users_list()
5     else:
6         return "Method not allowed"
7
8 # Bom
9 def lista_usuarios():
10     if request.method == 'GET':
```

Interativo

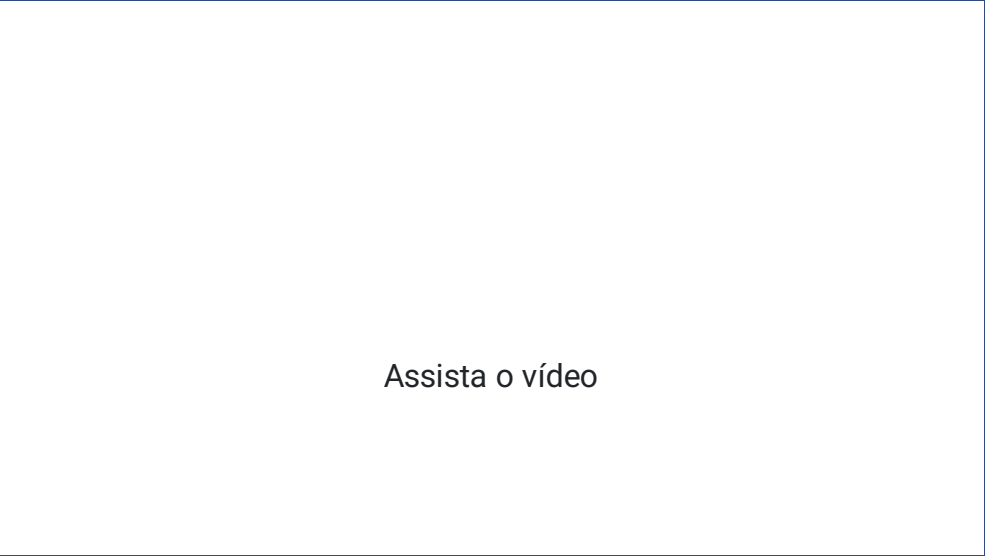
Descrição do interativo

Para reforçar os conceitos apresentados, vamos ver alguns exemplos de código.

Introdução à orientação de objetos



Neste vídeo, você verá alguns exemplos comentados de código em Python, e iremos aplicar algumas das boas práticas que estudamos nesta aula.



Assista o vídeo

Referências

Clique aqui para acessar as referências e créditos desta aula.

